P3568R0

`break label;` and `continue label;`

Published Proposal, 2025-01-12

This version:
https://eisenwave.github.io/cpp-proposals/break-continue-label.html

Authors:
Jan Schultke
Sarah Quiñones

Audience:
SG22, SG17

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
eisenwave/cpp-proposals

Abstract

Introduce `break label` and `continue label` to `break` and `continue` out of nested loops and `switch`es as accepted into C2y, and relax label restrictions.

Table of Contents

§ 1. Revision history

This is the first revision.

§ 2. Introduction

While C++ already has a broad selection of control flow constructs, one construct commonly found in other languages is notably absent: the ability to apply `break` or `continue` to a loop or `switch` when this isn't the innermost enclosing statement. This feature is popular, simple, and quite useful:

Specifically, we propose the following functionality:

```
outer: for (auto x : xs) {
    for (auto y : ys) {
        if (/* ... */) {
            continue outer; // OK, continue applies to outer for loop
            break outer;    // OK, break applies to outer for loop
        }
    }
}

switch_label: switch (/* ... */) {
    default: while (true) {
        if (/* ...*/) {
            break switch_label; // OK, break applies to switch, not to whi
        }
```

```
    }
}

break outer;       // error: cannot break loop from the outside
goto outer;        // OK, used to be OK, and is unaffected by this proposa

switch_label:;     // OK, labels can be reused
goto switch_label; // error: jump target is ambiguous
```

The `break label` and `continue label` syntax is identical to that in [N3355] and has been accepted into C2y (see working draft at [N3435]). We bring that syntax into C++ and relax restrictions on labels to make it more powerful, and to address concerns in a rival proposal [N3377].

Note that `break` and `continue` with labels have been proposed in [N3879] and rejected at Rapperswil 2014 ([N4327]):

> Straw poll, proposal as a whole:
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 1 | 1 | 1 | 13 | 10 |
>
> "break label;" + "continue label;"
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 3 | 8 | 4 | 9 | 3 |

Ten years later, circumstances have changed and we should re-examine this feature.

2.1. What has changed since 2014

The acceptance of the feature into C2y justifies re-examination in itself.

Furthermore, use of `constexpr` has become tremendously more common, and `goto` may not be used in constant expressions. Where `goto` is used to break out of nested loops, `break label` makes it easy to migrate code:

EXAMPLE 1

Uses of `goto` to break out of nested loops can be replaced with `break label` as follows:

```
constexpr void f() {
    outer: while (/* ... */) {
        while (/* ... */) {
            if (/* ... */) {
                goto after_loop;
                break outer;
            }
        }
    }
    after_loop:;
}
```

Last but not least, `break label` and `continue label` have seen an increase in popularity over the last ten years. The feature has seen increased adoption in older languages such as JavaScript, and newer languages have been popularized which support this construct, such as Rust and Kotlin.

Nowadays, it seems almost unthinkable not to have such control statements in a new language. A recent example of this is [Cpp2] (cppfront), which has `break label;` and `continue label;`.

3. Motivation

`break label` and `continue label` are largely motivated by the ability to control nested loops. This is a highly popular feature in other languages, and C++ could use it too, since it has no good alternative.

To be fair, a conditional `return` in the loop sometimes bypasses the need to terminate it. However, this is not always allowed; such practice is outlawed by MISRA-C++:2008 Rule 6-6-5 "A function shall have a single point of exit at the end of the function" ([MISRA-C++]). Even if it is permitted, there are many cases where an early `return` does not obsolete `break`, and it generally does not obsolete `continue`.

3.1. No good alternative

Let’s examine a motivating example which uses our new construct:

EXAMPLE 2 ¶

```
void f() {
    process_files: for (const File& text_file : files) {
        for (std::string_view line : text_file.lines()) {
            if (makes_me_angry(line)) {
                continue process_files;
            }
            consume(line);
        }
        std::println("Processed {}", text_file.path());
    }
    std::println("Processed all files");
}
```

`continue label` is very useful in this scenario, and expresses our intent with unparalleled clarity. We want to continue processing other files, so we `continue process_files`.

A plain `break` cannot be used here because it would result in executing the following `std::println` statement, but this should only be done upon success.

There are alternative ways to write this, but all of them have various issues.

§ 3.1.1. `goto`

```
for (const File& text_file : files) {
    for (std::string_view line : text_file.lines()) {
        if (makes_me_angry(line)) {
            goto done_with_file;
        }
        consume(line);
    }
    std::println("Processed {}", text_file.path());
    done_with_file:
}
std::println("Processed all files");
```

`goto` is similar in complexity and even readability here, however there are some issues:

- `goto` cannot cross (non-vacuous) initialization, which would be an issue if some variable was initialized prior to `std::println`. This can be addressed by surrounding the outer loop contents with another set of braces, but this solution isn't obvious and takes away from the elegance of `goto` here.
- `goto` cannot be used in constant expressions. For processing text files like in the example, this doesn't matter, but nested loops are desirable in a `constexpr` context as well.
- Many style guides ban or discourage the use of `goto`. See [MISRA-C++], [CppCoreGuidelinesES76], etc. This discouragement dates all the way back to 1968 (see [GotoConsideredHarmful]), and 66 years of teaching not to use `goto` won't be undone.
- Even in the cases where `goto` isn't discouraged, those cases are always special, like "only `goto` forwards", "only `goto` to break out of loops", etc.. This issue has been debated for decades, and there is still no consensus on when, actually, `goto` is okay to use.
- `goto` is innately more difficult to use because to understand its purpose, the user has to know where the jump target is located. A `goto past_the_loop` behaves radically differently compared to a `goto before_the_loop`. Moving the jump target or the `goto` statement relative to each other can also completely change these semantics. By comparison, `break` and `continue` always jump forwards, past a surrounding loop, or to the end of a surrounding loop respectively. This makes them much easier to reason about, and much less error-prone.
- The "local readability" of `goto` relies heavily on high-quality naming for the label. A `goto end` could mean to the end of a loop, to after the loop, to the end of a function, etc. Since `break` and `continue` are much more limited, they do not require such good label naming. A `break loop` has bad name, but the user *generally* understands its purpose.

> NOTE: Previous discussion on the [isocpp-core] reflector has addressed the idea of just adding `constexpr goto`, but doing so is alleged to be more complicated than more limited `constexpr` control flow structures which can only "jump forwards", such as `break` and `continue`.

In conclusion, there are too many issues with `goto`, some of which may never be resolved. [std-proposals] discussion prior to the publication of this proposal has shown once again that `goto` is a controversial and divisive.

§ 3.1.2. Immediately invoked lambda expression (IILE)

```
for (const File& text_file : files) {
    [&] {
        for (std::string_view line : text_file.lines()) {
```

```
            if (makes_me_angry(line)) {
                return;
            }
            consume(line);
        }
        std::println("Processed {}", text_file.path());
    }();
}
std::println("Processed all files");
```

While this solution works in constant expressions, we may be painting ourselves into a corner with this design. We cannot also `break` the surrounding loop from within the IILE, and we cannot return from the surrounding function. If this is needed at some point, we will have to put substantial effort into refactoring.

Furthermore, this solution isn't exactly elegant:

- The level of indentation has unnecessarily increased through the extra scope.
- The call stack will be one level deeper during debugging. This may be relevant to debug build performance.
- The fact that the lambda is immediately invoked isn't obvious until reading up to `()`.
- The word `return` does not express the overall intent well, which is merely to continue the outer loop. This can be considered a teachability downside.

It is also possible to use an additional function instead of an IILE in this place. However, this is arguably increasing the degree of complexity even more, and it scatters the code across multiple functions without any substantial benefit.

§ 3.1.3. Mutable `bool` state

```
for (const File& text_file : files) {
    bool success = true;
    for (std::string_view line : text_file.lines()) {
        if (makes_me_angry(line)) {
            success = false;
            break;
        }
        consume(line);
    }
```

```
    if (success) {
        std::println("Processed {}", text_file.path());
    }
}
std::println("Processed all files");
```

This solution substantially increase complexity. Instead of introducing extra scope and call stack depth, we add more mutable state to our function. The original intent of "go process the next file" is also lost.

Such a solution also needs additional state for each nested loop, i.e. two `bool`s are needed to `continue` from a loop "two levels up".

§ 3.2. Argumentum ad populum

Another reason to have `break label` and `continue label` is simply that it's a popular construct, available in other languages. When Java, JavaScript, Rust, or Kotlin developers pick up C++, they may expect that C++ can `break` out of nested loops as well, but will find themselves disappointed.

[StackOverflow] *"Can I use break to exit multiple nested `for` loops?"* shows that there is interest in this feature (393K views at the time of writing).

A draft of the proposal was posted on [Reddit] and received overwhelmingly positive feedback (70K views, 143 upvotes with, 94% upvote rate at the time of writing).

§ 3.2.1. Poll

Another way to measure interest is to simply ask C++ users. The following is a committee-style poll (source: [TCCPP]) from the Discord server Together C & C++, which is the largest server in terms of C++-focused message activity:

> Should C++ have "break label" and "continue label" statements to apply break/continue to nested loops or switches?

SF	F	N	A	SA
21	21	12	6	4

NOTE: 64 users in total voted, and the poll was active for one week.

§ 3.2.2. How common is `break`/`continue` with labels?

To further quantify the popularity, we can use GitHub code search for various languages which already support this feature. The following table counts only control statements with a label, *not* plain `break;`, `continue;`, etc.

Language	Syntax	Labeled `break`s	Labeled `continue`s	Σ `break` `continue`	`goto`s
Java	`label: for (...) break label; continue label;`	424K files	152K files	576K files	N/A
JavaScript	`label: for (...) break label; continue label;`	53.8K files	68.7K files	122.5K files	N/A
Perl	`label: for (...) last label; next label;`	34.9K files	31.7K files	66.6K files	16.9K files
Rust	`label: for (...) break 'label; continue 'label;`	30.6K files	29.1K files	59.7K files	N/A
TypeScript	`label: for (...) break label; continue label;`	11.6K files	9K files	20.6K files	N/A
Swift	`label: for ... break label continue label`	12.6K files	5.6K files	18.2K files	N/A
Kotlin	`label@ for (...) break@label continue@label`	8.7K files	7.6K files	16.3K files	N/A
D	`label: for (...)`	3.5K files	2.6K files	6.1K files	12.3K files

	`break label;` `continue label;`				

Go	`label: for ...`
	`break label;`
	`continue label;`
		270 files	252 files	522	1.5K files

Cpp2 (cppfront)	`label: for ...`
	`break label;`
	`continue label;`
		N/A	N/A	N/A	N/A

C	`label: for (...)`
	`break label;`
	`continue label;`
		N/A	N/A	N/A	7.8M files

Based on this, we can reasonably estimate that there are at least one million files in the world which use labeled `break`/`continue` (or an equivalent construct).

> NOTE: The `break` and `continue` columns also count equivalent constructs, such as Perl's `last`.

> NOTE: This language list is not exhaustive and the search only includes open-source code bases on GitHub.

> NOTE: The D `goto` count is inflated by `goto case;` and `goto default;` which perform `switch` fallthrough.

> NOTE: Fun fact: `continue` occurs in 5.8M C files, meaning that `goto` is more common.

§ 3.3. C2y compatibility

Last but not least, C++ should have `break label` and `continue label` to increase the amount of code that has a direct equivalent in C. Such compatibility is desirable for two reasons:

- `inline` functions or macros used in C/C++ interoperable headers could use the same syntax.
- C2y code is much easier to port to C++ (and vice-versa) if both languages support the same control flow constructs.

Furthermore, the adoption of [N3355] saves EWG a substantial amount of time when it comes to debating the syntax; the C++ syntax should certainly be C-compatible.

> NOTE: The [N3355] syntax is still subject to discussion; see § 4.4 What about syntax?.

§ 4. Design Considerations

§ 4.1. Alternative `break` and `continue` forms

There are some alternative forms of `break` and `continue` from various proposals/discussions, not just `break label;`. None of these are proposed by [N3355] or [N3377], which is reason enough to reject them. We discuss these regardless for the sake of completion.

§ 4.1.1. `break N`

A possible alternative to `break label;` would be a `break N;` syntax (analogous for `continue`), where `N` is an *integer-literal* or *constant-expression* which specifies how many levels should be broken. For example:

```
while (/* ... */)
    while (/* ...*/)
        break 2; // breaks both loops
```

We don't propose this syntax for multiple reasons. Firstly, [N3355] points out readability concerns, concerns when using `break N;` in a macro, and these are valid concerns in C++ as well.

Secondly, `break N` is more challenging to read because the developer has to investigate what scopes surround the statement (where e.g. `if` doesn't count, but `switch` and `for` count), and conclude from this where `break` applies. The greater `N` is, the ore challenging this task becomes. By comparison, `break label;` obviously breaks out of the loop labeled `label:`.

Thirdly, this construct is an obscure idea (not entirely novel, seen before in PHP). In our experience, obscure control flow ideas are unpopular and not worth pursuing. An extreme negative reaction to obscure control flow ideas was seen for the `goto default;` and `goto case X;` statements proposed in [N3879]. By comparison, `break label;` is completely mainstream; such code has likely been written a million times or more already (based on numbers in § 3.2.2 How common is break/continue with labels?).

4.1.2. `break while` et al.

Yet another novel idea has been suggested at [std-proposals-2]:

```
while (/* ... */) {
 for (/* ... */) {
   if (/* ... */) {
     break while; // break the while loop, not the for loop
     // break for while; // identical in functioning to the above version
    }
  }
}
```

This idea has been received negatively, and we strongly oppose it. It is not as obvious what the targeted statement is, as with `break N;`, and code can easily be broken by relocating the `break for while for;` or whatever statement somewhere else.

4.1.3. `break statement`

Perhaps the most exotic proposal is found in `break statement;` proposed in [P2635R0]. Such a statement would execute `statement` in the scope that has been entered by `break` or `continue`:

```
for (auto i : range_i) {
    for (auto j : range_j) {
        break continue; // breaks the inner loop, and continues the outer
    }
}
```

The author has seemingly abandoned that proposal, but even if they didn't, this idea is quite flawed.

EXAMPLE 3 ¶

`break return i;` could be used to `return` a shadowed entity `i` because name lookup of the *`statement`* takes place in the jumped-to scope:

```
float i;
for (int i = 0; i < 10; ++i) {
    break return i; // returns float i
}
```

The overarching issue is that statements don't always "belong" to the scope in which they appear syntactically; this is difficult to reason about, even if we don't add name lookup confusion like in the example.

§ 4.2. What about `break label;` for block statements etc.?

The following is *not* proposed:

```
label: {
    break label;
}
```

Being able to apply `break` or `continue` to additional constructs in C++ would be a controversial and novel idea. We simply want to apply `break` and `continue` to the same things you can already apply it to, but also state *which* construct they apply to, if need be.

However, the syntax we propose allows for such a construct to be added in the (distant) future; see § 4.4.3.8 Extendability.

§ 4.3. Should there be `break label;` and `continue label;` in constant expressions?

Yes, absolutely! This is a major benefit over `goto`, and it's part of the motivation for this proposal.

An implementation is also quite feasible, and *basically* already exists in every compiler. For constant evaluation, `break` already needs to be able to exit out of arbitrarily deeply nested scopes:

```
while (/* ... */) {
    if (/* ... */) {
        { { { { { break; } } } } }
    }
}
```

The only novelty offered by `break label;` is that additional surrounding scopes can be "skipped", which is simple to implement, both for constant expressions and regular code.

§ 4.4. What about syntax?

We strongly support the currently accepted syntax of [N3355]. This syntax is

- simple and intuitive,
- has been used in a variety of other languages, and
- is easy to implement, considering that labels already exist in that form.

It should be noted that there is a new competing proposal [N3377] for C2y, which instead proposes:

```
for outer (/* ...*/) {
    while (/* ... */) break outer;
    while outer (/* ... */) {
        // OK, applies to the enclosing while loop
    }
}
```

In summary, the competing syntax has the technical benefit that it doesn't require each `label:` to be unique within a function. This allows the developer to expand function-style macros containing labeled loops multiple times, and lets them repurpose simple names like `outer` and `inner` within the same function. We address these technical issues in § 4.5 Changes to labels, however, not with the [N3377] syntax.

[N3377] also makes largely more subjective claims as to why the `for label` syntax is a better fit for C, which we discuss below.

§ 4.4.1. Labels don’t properly imply the target

One claim made by [N3377] is that labels don’t properly imply a target and are their own declaration. This is actually true and based on the fact that a *compound-statement* consists of *block-item*s, and one of these can simply be a *label* with no *statement*.

However, these grammatical implementation details are easily changed and don’t matter all that much to the mental model of a programmer. The quintessential example of this is the preference of many C++ programmers for `int* x`, closely associating `*` with the `int` specifier, despite `*x` forming a *declarator* and thus being "more correct".

Similarly, `l:` is intuitively meant to attach to something else (based on the name "label" and on its syntax), even though the language grammar no longer mandates that. Whether `l: for` is a good syntax should depend on whether it expresses the idea of attaching a label to a `for` loop well, not on whether it is a perfect fit for the current grammar implementation details, which are subject to change anyway.

§ 4.4.2. Two sets of labels for disambiguation

Another benefit is that `goto` jump targets and loop names don’t share syntax, and this disambiguates code (with [N3377] syntax):

- When a developer sees `label:`, they know that this label is a jump target for `goto`.
- When a developer sees `for name`, they know that this `name` is a target for `break` or `continue`, and cannot be jumped to with `goto`.

For C, this is not a negligible concern. `goto` is slightly more common than `continue;` in C code on GitHub (source: § 3.2.2 How common is break/continue with labels?), but approx. half as common as `break;`. This means that for any `label:` with the [N3355] syntax in C, there is a decent chance that there are `goto`s nearby.

However, this problem is easy to overstate. Firstly, this ambiguity *only* exists for labeled loops, since arbitrary statements cannot be targeted by `break` or `continue`. For example, `label: free(pointer)` is obviously a `goto` target.

Secondly, we can make an educated guess about the purpose of a label in many situations:

- Labels towards the end of a function are likely targets for prior `goto`s which jump to cleanup/error handling code.

- Labels inside of loops are less likely to be `goto` targets because jumping into the middle of a is quite surprising, and may be illegal due to crossing initialization of a loop variable.
- Labels such as `end:`, `stop:`, `done:`, `success:`, `cleanup:`, `error:`, etc. heavily imply that earlier code uses `goto` to get there.
- Labels such as `loop:`, `outer:`, etc. heavily imply that code inside the loop uses `break label;` or `continue label;`.

4.4.2.1. Naming conventions as a workaround

Furthermore, disambiguation of `goto` targets and `break`/`continue` targets is possible through naming conventions for labels. For example, `break` and `continue` targets can be named `xzy_loop`, and such names can be avoided for `goto` jump targets.

Virtually every programming community already uses naming conventions for disambiguation. For example, method names conventionally use `camelCase` in Kotlin, and class names conventionally use `PascalCase`. This effectively disambiguates constructor calls from regular function calls for `F()`.

Naming conventions seem like a reasonable solution for disambiguating `goto` targets from `break` targets. We don't need to create two distinct label syntaxes to accomplish this. We can let people choose for themselves whether they want such disambiguation or not, which is much more in line with C and C++ design philosophy.

4.4.3. Strong opposition to N3377

We strongly oppose the N3377 syntax for multiple reasons, listed below.

4.4.3.1. Breaking precedent of most prior art

Most languages that supports both labeled loop control and `goto` statements have a single label syntax. [N3377] breaks this pattern.

EXAMPLE 4 ¶

Perl supports `goto LABEL`, `last LABEL`, and `next LABEL`, with shared label syntax:

```
goto LINE;
LINE: while (true) {
    last LINE;  # like our proposed break LINE
}
```

EXAMPLE 5 ¶

Go supports `goto Label`, `break Label`, and `continue Label`, with shared label syntax:

```
goto OuterLoop
OuterLoop: for {
    break OuterLoop
}
```

EXAMPLE 6 ¶

D supports `goto label`, `break label`, and `continue label`, with shared label syntax:

```
goto outer;
outer: while (true) {
    break outer;
}
```

The fact that none of these languages require separate syntax for `goto` targets and `break` targets proves that the syntax proposed by [N3377] is unnecessary, from a technical viewpoint. C is not so different from D or Go that this argument doesn't apply.

Such separate syntax would also be very surprising to Go, Perl, and D developers coming to C++ because they could reasonably expect `label:` to work for any kind of jump.

To be fair, opposite precedent also exists:

EXAMPLE 7 ¶

Ada supports `goto Label` with `<<Label>>`, and `exit label` with `Label:`:

```
goto Target;
<<Target>>
Outer: loop
    exit Outer; -- like our proposed break Outer
end loop Outer;
```

§ *4.4.3.2. Teachability, Simplicity, Elegance*

C and C++ have had the `label:` syntax for labeling statements for multiple decades now. It is extremely well understood, and has been replicated by other C-esque languages, such as Java, Rust, JavaScript, Kotlin, and more. Based on the numbers in § 3.2.2 How common is break/continue with labels?, we can assume that `label:`-like syntax has been used in over a million files already.

Now, decades after the fact, and a million files later, we need to invent our own, novel syntax *just* for labeling loops and `switch`es? No, we don't! Go, Perl, and D didn't need to either.

The [N3355] syntax can be intuitively understood at first glance, either through intuition from `goto` labels, or from prior experience with other languages. On the contrary, given the precedent set by `if constexpr`, the `for outer` syntax could mislead a user into believing that `outer` is some kind of contextual keyword. These first experiences matter.

§ *4.4.3.3. Reusable syntax in multi-language systems*

C and C++ do not exist in a vacuum. They are often being used to implement lower-level details of a larger system, written in another language (e.g. NumPy, which combines Python, C, and C++).

In such a multi-language system, it is highly beneficial to have common syntax because developers don't have to learn two entirely different languages, but rather, one and a half. With [N3355], C and C++ could have identical label syntax to JavaScript, Java, and other languages with which they are paired. [N3377] wastes this opportunity.

§ *4.4.3.4. Instant tooling integration*

Since the [N3355] syntax reuses existing `label:` syntax, it is immediately compatible with existing tooling, such as syntax highlighters, auto-formatters, and more.

§ *4.4.3.5. `do` ... `while`*

EXAMPLE 8 ¶

[N3377] proposes the following syntax for `do` ... `while` loops:

```
do {
    // ...
    break name;
    // ...
} while name(/* ... */);
```

This is consistent with `while` loops because the *block-name* is always placed after the `while` keyword. However, it also means that `break` and `continue` can apply to a *block-name* which has not yet appeared in the code. This is a readability issue; with the exception of `goto` and labels, function bodies can be understood by reading them from top to bottom.

NOTE: Of course, there are also entities that can be outside the function body, like other called (member) functions, types, etc. However, control flow and use of local entities generally follow a top-to-bottom structure.

EXAMPLE 9 ¶

In larger functions, this can also be incredibly disorienting:

```
while /* outer? */(true) {
    // ...
    // ...
    // ...
    do {
        while (true) {
            // ...
            if (condition)
                break outer; // <<< you are here
            // ...
        }
        // ...
        // ...
        // ...
    } while /* outer? */(true);
}
```

When starting to read this code from the middle (perhaps after jumping to a specific line in there), the reader doesn't even know whether they should look further up, or further down when searching for the loop labeled `outer`.

To be fair, `do` ... `while` loops are relatively rare, so assuming that the *block-name* can be found above is usually correct. However, it is not always correct, and that makes this syntax less ergonomic.

EXAMPLE 10 ¶

On the contrary, the [N3355] (and our) syntax *never* have `break` apply to a name which appears later:

```
name: do {
    // ...
    break name;
    // ...
} while (/* ... */)
```

The [N3377] syntax could be adjusted to place the *block-name* after `do`, but every way to proceed has downsides:

- Either `break` can refer to a *block-name* which appears arbitrarily far below, or
- the *block-name* is not always placed after `while`, which may be hard to teach and remember, or
- the *block-name* can be placed both after `while` and after `do`, which creates to competing styles for the same construct.

On the contrary, [N3355] has no such issues.

§ *4.4.3.6. Code pronunciation*

EXAMPLE 11 ¶

The [N3377] syntax interferes with how code is pronounced, from left to right:

```
// "(loop named outer) While x is greater or equal to zero:"
outer: while (x >= 0) { /* ... */ }

// "While (loop named outer) x is greater or equal to zero:"
while outer(x >= 0) { /* ... */ }
```

Putting the loop name between the conjunction `while` and the dependent clause `x >= 0` is not easily compatible with the English language. A preceding label is less intrusive and doesn't need to be (mentally) pronounced, like a leading attribute, line number, paragraph number, etc.

This is not just a stylistic argument; it's an accessibility argument. C++ developers who rely on screen readers cannot "skip over" or "blend out" the `outer` like a sighted developer, and benefit from code that is more naturally pronounceable.

§ *4.4.3.7. Repetition*

In the event that a user wants to be `break` out of a loop and `goto` it, in the same function, repetition is needed:

```
goto outer;
// ...
outer: while outer(true) {
    while(true) {
```

```
            break outer;
        }
    }
```

Since traditional labels are entirely separate from the loop names, we need to specify the `outer` name twice here. Some people may consider it a benefit to keep loop names strictly separate from jump targets, however, we see it as detrimental:

- If the label is the same as the loop name, we repeat ourselves.
- Otherwise, we refer to the same statement using two different names, which feels disorienting. If we have a good, meaningful label for a loop, such as `main_event_loop:`, why shouldn't we also write `while main_event_loop`?

§ *4.4.3.8. Extendability*

Consider the following (not proposed) construct:

```
label: {
    break label;
}
```

> NOTE: C++ allows you to emulate such `break`s with `goto past_the_block;`, but see § 3.1.1 goto.

Other mainstream languages already have such a feature:

- That code above is valid Java, JavaScript, and TypeScript.
- When replacing `label` with `'label`, the code is valid Rust.
- Scala has `breakable` blocks in which `break()` can be used the same way.

Breaking a *block-statement* is currently *not* proposed, however:

- What if someone wants to propose `break`ing block statements in the future? There is no obvious [N3377]-like syntax for adding an identifier to a *block-statement*. Are we really certain that we will never want to do this, in the next 50 years?
- What if someone wants to add such `break`s as a compiler extension right now? With the [N3377] syntax, this wouldn't make sense, considering that "`goto` labels" cannot be addressed

by `break` at all.

- What about new statement that we want to be `break`able? Every new statement would need some unique way of adding a name, instead of us being able to use `label:` in every case.

On the contrary, the [N3355] syntax makes no such problematic commitments, and is easily compatible with

- existing language extensions like "GCC computed `goto`",
- every new statement in the future, and
- new language extensions, like `break` for block-statements.

It should be noted that [N3377] also floats the idea of an alternative Rust-like syntax, such as `'label: for`. This would no longer interfere with future standardization, but we don't believe that this is worth pursuing because it creates two extremely similar label syntaxes. It is almost impossible to justify why the language needs two almost identical label syntaxes when other languages (e.g. D, Go, Perl) have shown that this is technically unnecessary.

Such a strategy would also turn the prior example in § 4.4.3.7 Repetition into `outer: 'outer: for (/* ... */)`, which looks even worse.

§ *4.4.3.9. Blocking contextual keywords*

Considering that the user is able to add arbitrary identifiers after `while` and `for`, this makes it impossible to add future contextual keywords in that place without potentially breaking code:

```
while parallel(/* ... */)
```

If `parallel` was used as a label here, that may be broken by "parallel while loops" in the future. There is precedent for such changes in C++, in the form if `if constexpr`. To be fair, `constexpr` is a true keyword, so the addition of `constexpr` after `if` wouldn't have been blocked by [N3377] syntax either (if [N3377] was part of C++ at the time).

Nonetheless, it is substantial commitment to block contextual keywords with the [N3377] syntax, and we don't see that commitment as justified.

To be fair, [N3377] also floats the idea of a syntax such as:

```
while :outer:(/* ... */)
```

The addition of special characters into the *block-name* no longer blocks future contextual keywords. However, if we are requiring this, it seems strictly better to have a syntax such as

```
:outer: while
```

The only other language (to my knowledge) with two syntaxes is Ada (`<<Label>>` and `Label:`), but even that language puts the labels *before* the statement to which they apply, not somewhere in the middle, let alone at the end (in the case of `do while`).

A preceding syntax simply works better when considering *block-statement*s (§ 4.4.3.8 Extendability) and `do while` loops (§ 4.4.3.4 Instant tooling integration). However, once again, this unnecessarily gives us two almost identical label syntaxes.

§ *4.4.3.10. Labeling loops expanded from macros*

Because [N3355] loop labels are prepended to the loop, they can also be applied to loops expanded from macros. Such macro-expanded loops are relatively common in C.

EXAMPLE 12 ¶

The `HASH_ITER` macro from uthash expands to a for loop; see [UthashDocs].

```
#define HASH_ITER(hh,head,el,tmp)
for(((el)=(head)), ((*(char**)(&(tmp)))=(char*)((head!=NULL)?(head)->hh.
  (el) != NULL; ((el)=(tmp)), ((*(char**)(&(tmp)))=(char*)((tmp!=NULL)?(
```

The [N3355] syntax lets the user `break` out of a `HASH_ITER` loop as follows:

```
struct my_struct *current_user, *tmp;

outer: HASH_ITER(hh, users, current_user, tmp) {
    for (/* ... */) {
        if (/* ... */) break outer;
    }
}
```

The [N3377] syntax makes it impossible to apply labels to existing such loop macros. To add a *block-name*, cooperation from the library author is needed.

NOTE: This argument is not so important to C++ because such loops would idiomatically be written as a function template containing a loop; instead, this argument is targeted towards C developers, who cannot use templates.

§ 4.5. Changes to labels

[N3377] points out legitimate issues with reusing the `label:` syntax (see § 4.4 What about syntax?). However, as stated, we strongly oppose the proposed [N3377] syntax, and we propose to make changes to label semantics instead. These changes keep the syntax the same as [N3355].

First and foremost, we permit the same `label:` multiple times within the same function, see § 4.5 Changes to labels.

EXAMPLE 13 ¶

```
outer: while (true) {
    inner: while (true) {
        break outer; // breaks enclosing outer while loop
    }
}

outer: while (true) { // OK, reusing label is permitted
    inner: while (true) {
        break outer; // breaks enclosing outer while loop
    }
}

goto outer; // error: ambiguous jump target
```

NOTE: This code is well-formed Java and JavaScript. When using the labels `'outer` and `'inner` instead, this code is also well-formed Rust.

In other words, we are doubling down on the [N3355] syntax and changing labels to behave more like other mainstream languages.

§ 4.5.1. New labels - `goto` issues

The label changes have some implications for `goto`:

```
x: f();
x: g();
goto x; // error: jump is ambiguous
```

Labeling multiple statements with `x:` would now be permitted. Even though this is essentially useless considering that `f()` and `g()` are not loops, it makes the rules easier to teach, and easier to understand; there are no special rules for loops.

`goto x;` is ill-formed because it is ambiguous which `x:` label it is meant to jump to. This change doesn't break any existing code because existing code cannot have such ambiguities.

§ 4.5.2. New labels - what about nesting?

Another case to consider is the following:

```
l: while (true) {
    l: while (true) {
        break l;
    }
}
```

> NOTE: This code is not valid Java or JavaScript, but is valid Rust when using the label `'l`.

We believe that this code should be well-formed. Developers may run into this case when nesting pairs of `outer:`/`inner:` loops in each other "manually", or when a `l:` labeled loop in a macro is expanded into a surrounding loop that also uses `l:`.

Such cases are the motivation for [N3377], and should be addressed. [N3355] does not currently permit such nesting, and that fact will have to be resolved somehow, either by significant syntax changes through [N3377], or through relaxation of label rules.

§ 4.5.3. New labels - what about direct duplicates?

A more extreme form of the scenario above is:

```
l: l: l: l: f();
```

We also believe that this code should be well-formed because it's not harmful, and may be useful in certain, rare situations.

> EXAMPLE 14 ¶
> A somewhat common C idiom is to expand loops from macros; see also § 4.4.3.10 Labeling loops expanded from macros.
>
> ```
> outer: MY_LOOP_MACRO(/* ... */) {
>     break outer;
> }
> ```
>
> If `MY_LOOP_MACRO` already uses an `outer:` label internally, perhaps because it expands to two nested loops and uses `continue outer;` itself, then the macro effectively expands to `outer: outer:`.
>
> This forces the user to come up with a new label now, for a seemingly arbitrary reason.

Permitting this case has the benefit that *no code at all* can become ill-formed through applying labels. This rule is simple, teachable, and easy to implement.

§ 4.5.4. New labels - what about `break label` for loops with more than one label?

Another case to consider is this:

```
x: y: while (true) {
    break x;
}
```

Grammatically, `x: y: ...` is a *labeled-statement*, where the *statement* is another *labeled-statement* `y: ...`, with a *label* `y` and a *statement* `while ...`. In other words, `x:` doesn't even apply directly to the loop.

[N3355] makes wording changes specifically to address this, and to make this well-formed. So are we; this code should well-formed if only for the sake of C2y compatibility.

§ 5. Impact on existing code

No existing code becomes ill-formed or has its meaning altered. This proposal merely permits code which was previously ill-formed, and relaxes restrictions on the placement of labels.

§ 6. Implementation experience

An LLVM implementation is W.I.P.

A GCC implementation of [N3355] has also been committed at [GCC].

§ 7. Proposed wording

The wording is relative to [N5001].

Update [stmt.label] paragraph 1 as follows:

> A label can be added to a statement or used anywhere in a *compound-statement*.
>
> *label*:
>
> *attribute-specifier-seq*$_{\text{opt}}$ *identifier* `:`
>
> *attribute-specifier-seq*$_{\text{opt}}$ `case` *constant-expression* `:`
>
> *attribute-specifier-seq*$_{\text{opt}}$ `default :`
>
> *labeled-statement*:
>
> *label statement*
>
> The optional *attribute-specifier-seq* appertains to the label. ~~The only use of a label with an *identifier* is as the target of a `goto`. No two labels in a function shall have the same identifier.~~ A label can be used in a `goto` statement ([stmt.goto]) before its introduction.
>
> [*Note*: Multiple identical labels within the same function are permitted, but such duplicate labels cannot be used in a `goto` statement. — *end note*]

In [stmt.label] insert a new paragraph after paragraph 1:

> A label **L** of the form *attribute-specifier-seq*$_{\text{opt}}$ *identifier* `:` *labels* a *statement* **S** if
>
> - **L** is the *label* and **S** is the *statement* of a *labeled-statement* **X**, or
> - **L** labels **X** (recursively).
>
> [*Example*:
>
> ```
> a: b: while (0) { }            // both a: and b: label the loop
> c: { d: switch (0) {           // unlike c:, d: labels the switch stateme
>     default: while (0) { }     // default: labels nothing
> } }
> ```
>
> — *end example*]

NOTE: This defines the term *(to) label*, which is used extensively below. We also don't want `case` or `default` labels to label statements, since this would inadvertently permit `break i` given `case i:`, considering how we word [stmt.break].

Update [stmt.label] paragraph 3 as follows:

> A *control-flow-limited statement* is a statement **S** for which:
>
> - a `case` or `default` label appearing within **S** shall be associated with a `switch` statement ([stmt.switch]) within **S**, and
> - a label declared in **S** shall only be referred to by a statement ~~([stmt.goto])~~ in **S**.

NOTE: While the restriction still primarily applies to `goto` (preventing the user from e.g. jumping into an `if constexpr` statement), if other statements can also refer to labels, it is misleading to say "statement ([stmt.goto])" as if `goto` was the only relevant statement.

✧ ✧ ✧

Update [stmt.jump.general] paragraph 1 as follows:

> Jump statements unconditionally transfer control.
>
> *jump-statement*:
>
> `goto` *identifier* `;`
>
> `break` *identifier*$_{\text{opt}}$ `;`
>
> `continue` *identifier*$_{\text{opt}}$ `;`
>
> `return` *expr-or-braced-init-list*$_{\text{opt}}$ `;`
>
> ~~`goto` *identifier* `;`~~

> NOTE: `goto` is being relocated to the top so that all the jump statements with an *identifier* are grouped together. Of these three, `goto` is being listed first because it models the concept of "jumping somewhere" most literally; every following statement is more sophisticated or even defined as equivalent to `goto` (in the case of `continue`).

✧ ✧ ✧

Update [stmt.break] paragraph 1 as follows:

A *breakable statement* is an *iteration-statement* ([stmt.iter]) or a `switch` statement ([stmt.switch]). A `break` statement shall be enclosed by ([stmt.pre]) a breakable statement ~~an *iteration-statement* ([stmt.iter]) or a `switch` statement ([stmt.switch])~~ . If present, the *identifier* shall be part of a label **L** which labels ([stmt.label]) an enclosing breakable statement. The `break` statement causes termination of : ~~the smallest such enclosing statement;~~

- if an *identifier* is present, the smallest enclosing breakable statement labeled by **L**,
- otherwise, the smallest enclosing breakable statement.

~~control~~ Control passes to the statement following the terminated statement, if any.

[*Example*:

```
a: b: while (/* ... */) {
    a: a: c: for (/* ... */) {
        break;              // OK, terminates enclosing for loop
        break a;            // OK, same
        break b;            // OK, terminates enclosing while loop
        y: { break y; }     // error: break does not refer to a breakable
    }
    break c;                // error: break does not refer to an enclosi
}
break;                      // error: break must be enclosed by a breakab
```

— *end example*]

✧ ✧ ✧

Update [stmt.cont] paragraph 1 as follows:

A `continue` statement shall be enclosed by ([stmt.pre]) an *iteration-statement* ([stmt.iter]). If present, the *identifier* shall be part of a label **L** which labels ([stmt.label]) an enclosing *iteration-statement*. The `continue` statement causes control to pass to the loop-continuation portion of : ~~the smallest such enclosing statement, that is, to the end of the loop.~~

- if an *identifier* is present, the smallest enclosing *iteration-statement* labeled by **L**,
- otherwise, the smallest enclosing *iteration-statement*.

More precisely, in each of the statements

```
label: while (foo) {
  {
    // ...
  }
contin: ;
}

label: do {
  {
    // ...
  }
contin: ;
} while (foo);

label: for (;;) {
  {
    // ...
  }
contin: ;
}
```

~~a `continue` not contained in an an enclosed iteration statement is equivalent to `goto contin`.~~ the following are equivalent to `goto contin`:

- A `continue` not contained in an an enclosed iteration statement.
- A `continue label` not contained in an enclosed iteration statement labeled `label:`.

NOTE: The clarification "that is, to the end of the loop" was dropped entirely based on community feedback. "the end of the loop" is not all that much clearer either, and the whole `goto` equivalence portion should make it clear enough what the behavior is.

✧ ✧ ✧

Update [stmt.goto] paragraph 1 as follows:

> The `goto` statement unconditionally transfers control to ~~the~~ a statement labeled ([stmt.label]) by ~~the identifier~~ a label in the current function containing *identifier*, but not to a `case` label . ~~The identifier shall be a label located in the current function.~~ There shall be exactly one such label.

> NOTE: This wording has always been defective and our proposal fixes this. The term "to label" was never defined, and the requirement that an identifier shall be a label is impossible to satisfy because a label ends with a `:`, and an *identifier* in itself would never match the *label* rule.
>
> The previous wording may have also allowed for `goto x;` to jump between two different functions both containing `x:` because while there has to be some label `x:` in the current function, we don't say that we jump to the `x:` in the current function specifically.

§ 8. Acknowledgements

I thank Sebastian Wittmeier for providing a list of languages that support both `goto` and `break`/`last` with the same label syntax. This has been helpful for writing § 4.4.3.1 Breaking precedent of most prior art.

I think Arthur O'Dwyer and Jens Maurer for providing wording feedback and improvement suggestions.

I thank the Together C & C++ community for responding to my poll; see [TCCPP].

§ References

§ Normative References

[N5001]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 17 December 2024. URL: https://wg21.link/n5001

§ Informative References

[Cpp2]

Herb Sutter. *Cpp2 documentation - Loop names, break, and continue*. URL: https://hsutter.github.io/cppfront/cpp2/functions/#loop-names-break-and-continue

[CppCoreGuidelinesES76]

CppCoreGuidelines contributors. *CppCoreGuidelines/ES.76: Avoid goto*. URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-goto

[GCC]

Jakub Jelinek. *c: Implement C2Y N3355 - Named Loops [PR117022]*. URL: https://gcc.gnu.org/git/gitweb.cgi?p=gcc.git;h=50f27896adb272b40ab03a56fd192e74789bef97

[GotoConsideredHarmful]

Edgar Dijkstra. *Go To Statement Considered Harmful*. 1968. URL: https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf

[ISOCPP-CORE]

CWG. *Discussion regarding continue vs. goto in constant expressions*. URL: https://lists.isocpp.org/core/2023/05/14228.php

[MISRA-C++]

MISRA Consortium Limited. *MISRA C++:2023*. URL: https://misra.org.uk/product/misra-cpp2023/

[N3355]

Alex Celeste. *N3355: Named loops, v3*. 2024-09-18. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3355.htm

[N3377]

Erich Keane. *N3377: Named Loops Should Name Their Loops: An Improved Syntax For N3355*. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3377.pdf

[N3435]

JeanHeyd Meneide; Freek Wiedijk. *ISO/IEC 9899:202y (en) — n3435 working draft*. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3435.pdf

[N3879]

Andrew Tomazos. *Explicit Flow Control: break label, goto case and explicit switch*. 16 January 2014. URL: https://wg21.link/n3879

[N4327]

Ville Voutilanen. *C++ Standard Evolution Closed Issues List (Revision R10)*. 21 November 2014. URL: https://wg21.link/n4327

[P2635R0]

Justin Cooke. *Enhancing the break statement*. 21 August 2022. URL: https://wg21.link/p2635r0

[Reddit]

Jan Schultke. *"break label;" and "continue label;" in C++*. URL: https://www.reddit.com/r/cpp/comments/1hwdskt/break_label_and_continue_label_in_c/

[StackOverflow]

Faken. *Can I use break to exit multiple nested 'for' loops?*. 10 Aug 2009. URL: https://stackoverflow.com/q/1257744/5740428

[STD-PROPOSALS]

Jan Schultke. *Bringing break/continue with label to C++*. URL: https://lists.isocpp.org/std-proposals/2024/12/11838.php

[STD-PROPOSALS-2]

Filip. *Improving break & continue*. URL: https://lists.isocpp.org/std-proposals/2024/11/11585.php

[TCCPP]

Poll at Together C & C++ (discord.gg/tccpp). URL: https://discord.com/channels/331718482485837825/851121440425639956/1318965556128383029

[UthashDocs]

Troy D. Hanson; Arthur O'Dwyer. *uthash User Guide: Deletion-safe iteration*. URL: https://troydhanson.github.io/uthash/userguide.html#deletesafe